ЛР 4. Твой код простыми словами

Музыкальная клавиатура с физическими кнопками: где читается матрица, как выбирается режим и как нажатие превращается в ноту.

Все пути - внутри `lab4`. Файлы `.c` из таблицы лежат в `Core/Src`, файлы `.h` - в `Core/Inc`. Номера в блоках - строки исходников на 03.11.2026; длинные строки перенесены.

Карта основных файлов

Файл	За что отвечает
`main.c`	Запускает устройства и постоянно вызывает `app_update`.
`app.c`, `app.h`	Объединяют всё приложение: TEST/PIANO, нажатия, музыку и сообщения.
`keypad.c`, `keypad.h`	Читают четыре строки матрицы, определяют SB1-SB12 и подтверждают нажатие.
`i2c_bus.c`, `i2c_bus.h`	Обмениваются байтами с расширителем клавиатуры; следят за завершением и ошибками.
`button.c`, `button.h`	Подтверждают нажатие боковой кнопки PC15 для смены режима.
`piano.c`, `piano.h`	Выбирают ноту и её длительность, ведут гамму. Музыкальная часть из ЛР 3.
`sound.c`, `sound.h`	Передают нужную частоту аппаратному таймеру TIM2 и выключают звук.

`serial.c`, `serial.h`
`uart_hal.c`, `uart_backend.h`
`uart_pins.c`
	Очереди сообщений, UART через HAL и настройка выводов связи.

`clock.c`, `clock.h` `board.h`	Настройка частот, номера выводов и основные значения для платы.
`gpio_driver.c` `gpio_driver.h`	Твой драйвер настройки, чтения и записи выводов контроллера.

Путь нажатия

SB → расширитель PCA9538 → `i2c_bus` → keypad → код кнопки 1-12 → app → `piano` → `sound`. Параллельно app готовит сообщения для терминала.

В `.h` описаны функции и данные, а в `.c` написаны их действия.

1. main.c и app.c: порядок работы

main запускает устройства один раз

Core/Src/main.c, строки 18-23

```
18      SystemCoreClockUpdate();
19      if (HAL_Init() != HAL_OK) fatal(1);
20      if (!clock_init()) fatal(2);
21      if (!sound_init()) fatal(3);
22      if (!serial_init(true)) fatal(4);
23      if (!app_init(&app, HAL_GetTick())) fatal(5);
```

Готовятся HAL и время, частоты, звук и UART с прерываниями. app_init создаёт состояние приложения: режим TEST, настройки пианино, боковую кнопку и сканер матрицы (app.c, 83-96).

Core/Src/main.c, строки 29-30

```
29      for (;;) {
30          if (!app_update(&app, HAL_GetTick())) fatal(6);
```

Дальше main постоянно вызывает app_update, передавая время в миллисекундах. Основные действия собраны именно в этой функции.

Что выполняет один вызов app_update

Строки в app.c	Действие
100-106	Обслужить UART и удалить до 16 входных байтов из терминала. Управление идёт от кнопок стенда.
107-116	Проверить PC15 и при новом нажатии переключить TEST/PIANO.
117-118	Обновить время музыки и сделать очередной шаг чтения клавиатуры.
119-131	Обработать новое нажатие: сообщить код либо передать его в piano.
132-145	Сообщить о новой ноте гаммы и изменении доступности клавиатуры.
146-148	Установить текущую частоту звука, продвинуть вывод сообщения и вернуть результат.

Какие данные хранит App

В Core/Inc/app.h, строки **18-27**: piano - музыка, button - PC15, keypad - матрица, mode - режим, reports - очередь сообщений. Там же находятся счётчики событий и ошибок вывода.

Ошибка звукового модуля возвращает false из app_update, после чего main вызывает fatal(6). Код ошибки хранится в lab4_fault. SysTick_Handler, строка 34, обновляет время.

2. app.c: режимы и назначение кнопок

PC15 переключает TEST и PIANO

Core/Src/app.c, строки 109-115

```
109     if (switched) {
110         app->mode = app->mode == APP_MODE_TEST ? APP_MODE_PIANO : APP_MODE_TEST;
111         ++app->mode_changes;
112         piano_stop(&app->piano);
113         app->reported_scale_note = PIANO_NO_NOTE;
114         keypad_rearm(&app->keypad, now_ms);
115         notify(app, REPORT_MODE, (uint8_t)app->mode);
```

button_update в строках 107-108 даёт одно подтверждённое нажатие PC15. Затем режим меняется, музыка останавливается через piano_stop, а keypad_rearm требует полного отпускания матрицы. Октава и длительность сохраняются.

В button.c, строки **7-18**, новое положение проверяется 30 мс. Удержание PC15 не создаёт повторных переключений. Знак ! при чтении нужен потому, что нажатая кнопка даёт 0.

Как код SB превращается в действие пианино

Core/Src/app.c, строки 16-19

```
16      static const uint8_t mapping[KEYPAD_KEY_COUNT] = {
17          '1', '2', '3', '4', '5', '6', '7', '+', '-', 'A', 'a', '\r'
18      };
19      return key >= 1U && key <= KEYPAD_KEY_COUNT ? mapping[key - 1U] : 0;
```

Это таблица app_key_character: код 1 выбирает первый символ, код 12 - последний. Так физические кнопки превращаются в команды, которые уже понимает piano_input из ЛР 3.

Кнопка	В режиме PIANO
SB1-SB7	До, ре, ми, фа, соль, ля, си: символы 1-7.
SB8 / SB9	Октава выше / ниже: + и -.
SB10 / SB11	Длительность больше / меньше: A и a.
SB12	Гамма из семи нот: символ Enter.

Где различаются два режима

В app.c, строки **119-124**: TEST ставит сообщение Key N в очередь; PIANO вызывает piano_input. Условие !switched не даёт исполнить матричное нажатие в том же проходе, где поменяли режим.

3. keypad.c: как читается матрица

PCA9538 - микросхема кнопок с адресом 0x71 (keypad.h, 15). keypad_update, строки 146-248, постепенно обходит четыре строки и собирает состояние всех 12 кнопок.

Подготовка расширителя

В строках **151-176** сначала записывается Configuration = 0xFF: все линии становятся входами. Затем Output = 0x00 и Polarity = 0x00: подготовлен нулевой выход и обычное чтение уровней.

Выбирается только одна строка

Core/Src/keypad.c, строки 181-183

```
181         start_write(keypad, PCA9538_CONFIG_REG,
182                     (uint8_t)(0xFFu & ~(1u << keypad->row)),
183                     KEYPAD_SCAN_ROW_WAIT, now_ms);
```

У выбранной строки сбрасывается бит Configuration: она становится выходом с 0. Остальные строки остаются входами. После завершения записи код ждёт 1 мс через проверку времени (185-193), затем читает регистр входов (195-200).

Строка	P4	P5	P6
P0	SB1	SB5	SB9
P1	SB2	SB6	SB10
P2	SB3	SB7	SB11
P3	SB4	SB8	SB12

Прочитанные столбцы добавляются в общую маску

Core/Src/keypad.c, строки 204-209

```
204             const uint8_t inputs = i2c_bus_data();
205             for (uint8_t column = 0u; column < KEYPAD_COLUMN_COUNT; ++column) {
206                 if ((inputs & (1u << (column + KEYPAD_ROW_COUNT))) == 0u) {
207                     keypad->frame_mask |= (uint16_t)(1u <<
208                         (keypad->row + KEYPAD_ROW_COUNT * column));
209                 }
```

Нулевой уровень столбца означает нажатие в выбранной строке. `row + 4 * column` даёт номер бита: бит 0 - SB1, бит 5 - SB6, бит 11 - SB12. `frame_mask` - набор таких признаков для всех кнопок.

Когда получается готовый результат

После четырёх строк, **217-228**, кадр сохраняется в `last_mask`, помечается исправным и передаётся в `keypad_filter_update`. Он возвращает подтверждённый код 1-12 или 0, если нового нажатия нет.

4. keypad.c: одно нажатие - одно событие

Фильтр находится в keypad_filter_update, строки **31-82**. На вход он получает целый кадр: mask - какие кнопки нажаты, valid - можно ли доверять чтению, now_ms - текущее время.

Ошибка кадра или несколько кнопок блокируют ввод

Core/Src/keypad.c, строки 34-38

```
34      if (!valid || mask > KEYPAD_KEY_MASK ||
35          (mask != 0u && (mask & (uint16_t)(mask - 1u)) != 0u)) {
36          /* Chords/errors must not leave a pending candidate or release timer. */
37          keypad_filter_rearm(filter, now_ms);
38          return 0u;
```

Это условие отбрасывает неверный кадр, лишние биты и несколько нажатых кнопок. keypad_filter_rearm сбрасывает ожидание и включает blocked. Смена одной кнопки на другую без устойчивого отпускания тоже блокирует ввод: строки 59-61.

Новое нажатие должно держаться 30 мс

Core/Src/keypad.c, строки 71-81

```
71      if (now_ms - filter->press_since_ms < KEYPAD_DEBOUNCE_MS) {
72          return 0u;
73      }
74
75      filter->delivered = true;
76      filter->press_tracking = false;
77      uint8_t code = 1u;
78      while ((mask >>= 1u) != 0u) {
79          ++code;
80      }
81      return code;
```

Пока 30 мс не прошли, возвращается 0. Потом delivered запоминает, что нажатие уже выдано, а цикл сдвигов вычисляет код кнопки по единственному установленному биту.

При удержании срабатывает проверка delivered в строках **64-66**: нового события нет. press_since_ms хранит начало проверки нажатия, release_since_ms - начало проверки отпускания.

Как разрешается следующее нажатие

Core/Src/keypad.c, строки 47-50

```
47          if (now_ms - filter->release_since_ms >= KEYPAD_DEBOUNCE_MS) {
48              filter->blocked = false;
49              filter->key_mask = 0u;
50              filter->delivered = false;
```

Эти строки выполняются, когда все кнопки отпущены. Через 30 мс снимается blocked и сбрасывается delivered. Только после этого можно принять новое нажатие. При включении и после смены режима фильтр тоже сначала ждёт полного отпускания.

Время 30 мс задано в Core/Inc/keypad.h, строка 13. Значение mask = 0 означает «вся матрица отпущена».

5. i2c_bus.c: обмен с расширителем

Данные обмена хранятся в bus, строки **9-16**: адрес, регистр, значение, этап phase и результат result. Обмен продвигает i2c_bus_service, которую вызывает keypad_update в строке 148.

Какие функции вызывает keypad.c

Функция и строки	Простой смысл
i2c_bus_init, 44-61	Включить I2C1, выводы шины и буфер через PD7.
i2c_bus_write_reg, 77-78	Начать запись значения в регистр расширителя.
i2c_bus_read_reg, 79-80	Начать чтение регистра расширителя.
i2c_bus_service, 162-255	Проверить текущее состояние и выполнить очередной шаг обмена.
i2c_bus_result, 81 i2c_bus_data, 82	Узнать результат и забрать прочитанный байт.

Что значит начать обмен

Core/Src/i2c_bus.c, строки 71-74

```
71      bus.started = now;
72      bus.phase = WAIT_FREE;
73      bus.result = I2C_BUS_PENDING;
74      return true;
```

Новый запрос принимается, если предыдущий не выполняется и адрес допустим. Параметры сохраняются, результат становится I2C_BUS_PENDING. Возврат true здесь означает «запрос принят»; завершение проверяется отдельно.

Где проходят этапы передачи

В switch внутри i2c_bus_service: **183-212** - дождаться свободной шины, передать адрес и номер регистра, затем значение при записи. **213-240** - завершить запись или начать чтение после повторного START.

Core/Src/i2c_bus.c, строки 243-246

```
243     case RECEIVE_BYTE:
244         if (status & I2C_SR1_RXNE) {
245             bus.received = (uint8_t)I2C1->DR;
246             bus.phase = STOP_FINISHED;
```

Байт читается из DR в bus.received. Затем строки 249-251 проверяют завершение STOP и ставят I2C_BUS_OK. До готовности нужного признака функция возвращается, а главный цикл продолжает работу.

Результаты IDLE, PENDING, OK, ERROR перечислены в Core/Inc/i2c_bus.h, строка 6.

6. Ошибки обмена и повторное подключение

Где ограничено время одного обмена

Core/Src/i2c_bus.c, строки 178-180

```
178     if ((uint32_t)(now_ms - bus.started) >= I2C_BUS_TIMEOUT_MS) {
179         fail(status, true, now_ms);
180         return;
```

Через 10 мс незавершённого обмена вызывается `fail` (85-94): увеличивает счётчики и начинает STOP. Проверка аппаратных ошибок - строки 174-177.

Где находится восстановление шины

Место в i2c_bus.c	Что делает
`recover`, 104-125	Даёт STOP до 2 мс на завершение. Если шина остаётся занятой, переводит линии в GPIO для восстановления.
128-159	Пытается дать девять импульсов SCL, затем сформировать STOP. Каждый шаг выполняется при следующих вызовах функции.
`recovery_finished`, 96-102	Возвращает выводы в AF4, заново настраивает I2C1 и сохраняет результат `I2C_BUS_ERROR`.

Строки **106-109** ограничивают восстановление 30 мс. Если линия остаётся зажатой внешним устройством, код выходит из попытки и сообщает об ошибке.

Что делает с ошибкой сканер клавиатуры

Core/Src/keypad.c, строки 97-103

```
 97      ++keypad->stats.errors;
 98      keypad->frame_mask = 0u;
 99      keypad->last_mask = 0u;
100      keypad->last_valid = false;
101      keypad->row = 0u;
102      (void)keypad_filter_update(&keypad->filter, 0u, false, now_ms);
103      transition(keypad, KEYPAD_SCAN_RETRY, now_ms);
```

`scan_error` отбрасывает неполный кадр, помечает данные неверными и блокирует нажатия. В ветке **231-241** сканер ждёт 50 мс после обнаружения ошибки и заново начинает подготовку расширителя. После восстановления кнопки нужно отпустить.

Где смотреть счётчики

`i2c_bus_stats()` - ошибки, отказы подтверждения и тайм-ауты. `keypad_stats()` - кадры, ошибки сканирования и выданные нажатия. В `app.c`, **137-145**, формируются сообщения об отключении и возвращении клавиатуры.

Пределы 10 и 30 мс - `i2c_bus.h`, 8-9; повтор через 50 мс - `keypad.h`, 14. Одиночный BERR обрабатывается отдельно в `i2c_bus.c`, 167-173: код очищает признак и продолжает обмен.

7. piano.c и sound.c: ноты и звук

В PIANO функция app_key_character выбирает символ, а piano_input выполняет действие. Начальные настройки: октава 5/9 и 1 с (piano.h, 16-20).

Основные места в piano.c

Строки	Что происходит
5-15	Таблица частот девяти октав и семи нот. 440000 миллигерц означает 440 Гц.
39-48	start_playback: начало ноты или гаммы с выбранными настройками.
65-113	piano_input: цифры запускают ноты; +, -, A, a меняют настройки; Enter запускает гамму.
115-139	piano_update: заканчивает ноту или выбирает следующую ноту гаммы по времени.
33-37	piano_stop: останавливает воспроизведение, сохраняя выбранные октаву и длительность.

При запуске сохраняется копия настроек

Core/Src/piano.c, строки 42-47

```
42      piano->active_octave = piano->selected_octave;
43      piano->active_duration_ms = piano->selected_duration_ms;
44      piano->note_started_at_ms = now_ms;
45      piano->active_note = note;
46      piano->playing = true;
47      piano->scale = scale;
```

В начале настройки selected_* копируются в active_*. Текущая нота и вся запущенная гамма используют эту сохранённую копию. Новая нота или SB12 начинает новое воспроизведение.

Где частота превращается в звук

Core/Src/sound.c, строки 54-59

```
54      uint32_t ticks = sound_period_ticks(frequency_millihz);
55      if (!ticks) return false;
56      if (frequency_millihz == current_frequency) return true;
57      sound_stop();
58      TIM2->ARR = ticks - 1U;
59      TIM2->CCR1 = ticks / 2U;
```

sound_period_ticks рассчитывает период. ARR получает период минус один, CCR1 - половину периода. Строки 60-64 запускают TIM2. Прежняя частота продолжает звучать без перенастройки; 0 выключает звук через строки 50-52.

В app.c, строка 146, частота из piano_frequency_millihz передаётся в sound_set_frequency. Границы октав и длительности находятся в Core/Inc/piano.h, строки 11-20.

8. app.c: очередь сообщений и UART

В reports помещается до **16 событий**. app превращает их в текст, а TX модуля serial вмещает до **512 байт** для отправки через UART.

notify добавляет событие

Core/Src/app.c, строки 24-26

```
24      if (app->report_count == APP_REPORT_CAPACITY) { ++app->dropped_reports; return; }
25      AppReport *r = &app->reports[app->report_head];
26      *r = (AppReport){.kind = kind, .key = key};
```

Если очередь полна, растёт dropped_reports. Иначе создаётся запись с данными события: нотой, частотой, октавой и длительностью (27-34). Поэтому задержанный вывод описывает состояние на момент события.

Какие сообщения формирует send_report

Событие	Вывод и строки в app.c
REPORT_MODE	Текущий режим: keyboard test или piano, 47-50.
REPORT_KEY	Код Key N в режиме TEST, 51-53.
REPORT_NOTE	Имя ноты, октава, частота и длительность, 54-59.
REPORT_SETTINGS	Новые выбранные настройки, 60-64.
REPORT_LIMIT	Достигнута граница настройки, 65-69.
REPORT_OFFLINE REPORT_ONLINE	Ошибка I2C или возвращение клавиатуры, 70-75.

Когда событие можно убрать из очереди

Core/Src/app.c, строки 77-80

```
77      if (size > 0 && (size_t)size < sizeof(text) && serial_write(text, (size_t)size)) {
78          app->report_tail = (uint8_t)((app->report_tail + 1U) % APP_REPORT_CAPACITY);
79          --app->report_count;
80      }
```

Только когда текст успешно передан в очередь serial_write, событие удаляется. Если в TX мало места, send_report ждёт следующих проходов: проверка запаса 192 байта стоит в строке 42.

Где находится фактическая отправка

serial.c: serial_write (21-34) кладёт байты в TX, serial_service (87-118) запускает передачу. uart_hal.c (51-76) сообщает о её завершении через serial_transmitted. Матрица и музыка обновляются и при задержке вывода.

Размеры очередей: app.h, 9; serial.h, 10. Ввод терминала удаляется в app.c, 103-106.

9. Настройки и вспомогательные файлы

Где лежат значения для платы

Что настроено	Файл и строки
Звук и UART	`Core/Inc/board.h`, 5-10: PA5/AF1, TIM2 60 МГц, UART 57600.
I2C и включение буфера	`Core/Inc/board.h`, 11-19: PB8/PB9, AF4, 80 кГц; включение через PD7.
Боковая кнопка	`Core/Inc/board.h`, 20-22: PC15, проверка нажатия 30 мс.
Матрица и её фильтр	`Core/Inc/keypad.h`, 11-15: 12 кнопок, 30 мс, повтор 50 мс, адрес 0x71.

Остальные файлы проекта

Файл или группа	Назначение

`Core/Src/clock.c`
`Core/Src/gpio_driver.c`
и соответствующие `.h`
	Частоты: ядро 120 МГц, APB1 30 МГц, APB2 и TIM2 60 МГц. GPIO-драйвер настраивает выводы; `io_configure` начинается на строке 45.

`Core/Startup/`
`Core/Src/system_stm32f4xx.c`
`Drivers/`
	Стартовый и системный код, CMSIS и HAL производителя. В стартовом файле вызов `main` находится в строке 100.

`Core/Inc/`
`stm32f4xx_hal_conf.h`
`STM32F427VGTX_FLASH.ld`
	Настройки библиотеки HAL и размещения программы в памяти.

`tools/build.py` `tools/board.py`, `SDK11M.cfg`	Сборка, прошивка и отладка через FT2232/JTAG. По умолчанию прошивается `build/lab4.elf`.

`tests/test_lab4.c`
`tests/run_tests.py`
`tests/test.ld`
	Проверки фильтра, I2C, матрицы, приложения и запуска. Python исполняет ARM-код в эмуляторе.

`.project`, `.cproject`
`README.md`, `docs/`,
`.gitignore`
	Настройки IDE, описание и отчёт, схема и протокол проверок, исключения для Git.

Сборка и прошивка из корня репозитория

```
python3 lab4/tools/build.py
python3 lab4/tools/board.py flash
```

Скрипт создаёт `lab4/build/lab4.elf` и `.bin`. В CubeIDE результат лежит в Debug или Release; для него нужен путь через `board.py flash --elf ...`. Начало C-проверок - `test_entry`, строка 44 в `tests/test_lab4.c`.

10. Что показать на защите

Один пример: SB6 в режиме PIANO

Сначала PC15 выбирает PIANO. Матрица полностью отпущена и готова принимать новое нажатие.

1. Сканер выбирает P1 и видит нажатие в столбце P5. В `frame_mask` ставится бит `1 + 4 × 1 = 5`: это SB6.
2. После устойчивого нажатия на полных кадрах в течение 30 мс фильтр возвращает код 6.
3. `app_key_character` превращает код в символ 6. `piano_input` выбирает ноту с номером 5 - ля.
4. При начальной октаве 5/9 получается 440000 миллигерц, то есть 440 Гц. `sound` задаёт TIM2: `ARR = 136363`, `CCR1 = 68182`.
5. Через 1 с нота завершается: частота становится 0 и звук выключается. Сообщение о ноте проходит через `reports` и TX UART.

Быстрая подсказка: где что найти

Нужно показать	Открой
Режимы и назначение SB1-SB12	`Core/Src/app.c`, 14-20, 107-129
Чтение четырёх строк и сборку кадра	`Core/Src/keypad.c`, 146-228
30 мс, отсутствие повторов, отпускание	`Core/Src/keypad.c`, 31-82
Запуск обмена / его пошаговое выполнение	`Core/Src/i2c_bus.c`, 63-83 / 162-255
Восстановление шины и повтор сканирования	`Core/Src/i2c_bus.c`, 85-160 `Core/Src/keypad.c`, 95-104, 231-241
Музыку / настройку звукового таймера	`Core/Src/piano.c`, 65-146 `Core/Src/sound.c`, 48-66
Очередь событий и вывод сообщений	`Core/Src/app.c`, 22-81

Как коротко объяснить своё решение

«Я добавил к музыкальной клавиатуре ввод от кнопок стенда. Драйвер I2C читает расширитель, сканер собирает матрицу, фильтр подтверждает одно нажатие. Приложение либо выводит код в TEST, либо запускает действие пианино в PIANO. Обмен, музыка и вывод сообщений продвигаются по шагам в главном цикле».

Памятка составлена по текущим исходникам `lab4`. Фрагменты взяты из файлов автоматически; исходный код при подготовке документа не менялся.